Parallel Processing Assignment 1 - Design Decisions Report

Assignment Report

January 25, 2026

1 Introduction

This report documents the design decisions and performance considerations for the parallel implementation of three core functions in the order book processing system: `updateDisplay`, `totalAmountTraded`, and `printOrderStats`. Each function has been optimized using OpenMP parallel programming techniques to improve performance over their sequential counterparts.

2 Function Analysis

2.1 updateDisplay Function

Design Decisions: The parallel implementation of `updateDisplay` employs a sophisticated multi-phase approach to maximize parallelization opportunities. The key design decision was to restructure the algorithm into three distinct phases: (1) data preprocessing and stock grouping, (2) parallel processing by stock ID, and (3) parallel snapshot file generation.

The algorithm first groups all order book entries by stock ID, creating a map from stock IDs to their corresponding entries. This preprocessing step enables the core parallel section where each thread processes a different stock independently using `#pragma omp parallel for`. Each thread maintains its own `StockInfo` state and tracks snapshot boundaries based on entry indices and frequency parameters.

The final phase parallelizes file output operations, where each snapshot is written to disk concurrently. This design eliminates data races by ensuring each thread operates on independent data structures and file handles. The use of a 2D vector `finalData[snapshots][stocks]` allows threads to write their results without synchronization overhead.

2.2 totalAmountTraded Function

Design Decisions: The `totalAmountTraded` function implements a straightforward parallel reduction pattern using OpenMP's built-in reduction clause. The design prioritizes simplicity and efficiency by leveraging the `#pragma omp parallel for reduction(+:total)` directive.

Each thread processes a subset of the order book entries independently, performing bit unstuffing, packet decoding, and local accumulation of trading amounts. The reduction clause handles the thread-safe accumulation of partial results into the final total. This approach is optimal for this embarrassingly parallel problem, as there are no data dependencies between iterations.

The function maintains numerical accuracy by using `int64_t` arithmetic to prevent overflow issues when multiplying order quantities by order values. The parallel overhead is minimal since each iteration performs substantial computational work (bit manipulation and arithmetic operations).

2.3 printOrderStats Function

Design Decisions: The `printOrderStats` function implements a thread-local data aggregation strategy followed by a sequential merge phase. The design creates separate `StockInfo` maps for each thread to avoid lock contention during the parallel processing phase.

Each thread maintains its own private map (`threadStockMaps[thread_id]`) and processes a subset of order book entries using `#pragma omp parallel for`. Threads independently update their local stock statistics without synchronization. After parallel processing completes, the main thread sequentially merges all thread-local maps into a single result map.

This design minimizes synchronization overhead while maintaining data consistency. The trade-off is increased memory usage (multiple maps) for improved parallel efficiency. The final file output is performed sequentially to ensure consistent formatting and avoid file I/O conflicts.

3 Performance Considerations

3.1 Load Balancing

All three functions utilize OpenMP's default static scheduling for the `parallel for` constructs. This choice is appropriate given the uniform computational complexity of processing individual order book entries.

3.2 Memory Access Patterns

The parallel implementations are designed to maximize cache locality. The `updateDisplay` function groups data by stock ID to improve spatial locality when processing related entries. The thread-local maps in `printOrderStats` reduce cache line conflicts between threads.

3.3 Scalability

The algorithms scale well with the number of available CPU cores. The reduction-based approach in `totalAmountTraded` provides near-linear speedup for large datasets. The data partitioning strategies in the other functions minimize inter-thread dependencies, enabling effective utilization of multiple cores.

4 Conclusion

The parallel implementations demonstrate effective use of OpenMP constructs to achieve significant performance improvements over sequential versions. The design decisions prioritize correctness, minimize synchronization overhead, and maximize parallel efficiency while maintaining the functional requirements of the order book processing system.